

PME Extensions: Gotchas + Tech Debt

In this document we detail facts (gotchas!) and tech debt which needs to be taken into account for future work on the Arrow and DBPS codebases.

C++17 vs C++20

At the time of developing the PME extensions, the C++ Arrow codebase was still using the C++17 standard of the language. Even when we performed the rebase of the Apache Arrow main codebase with the Protegrity fork (in November 2025), Arrow was still using C++17. For this reason, we also developed DBPS using C++17, in order to guarantee STL and binary compatibility of the shared libraries produced in DBPS to be loaded into the Arrow code.

The Apache Arrow mainline codebase has completed a migration to C++20 (as of release 23.0 in January 2026.).

Usage of `tcb::span`

The C++ `std::span` data structure which exists starting in C++20 provides a non-owning view over a contiguous byte sequence (pointer + length). It brings many benefits for code simplicity and memory management. However, `std::span` was not available for development given the usage of C++17 (mentioned above). For this reason, we decided to make use of [tcb::span](#), a back-port of the `std::span` compatible with C++11 and onwards.

`tcb::span` is included in [dbpa_interface.h](#), the header file which defines the `DataBatchProtectionAgent` interface and is consumed in the Arrow codebase.

In a future migration to C++20, the usage of `tcb::span` will likely need to be phased out.

Enum namespaces (DBPS vs Parquet)

The mental model when developing DBPS and the PME extensions was to create DBPA as an entity independent from Arrow and Parquet concepts, and for Arrow to take dependency on DBPA (and not the other way around).

However, one of the desired functionalities of DBPA is to perform per-value encryption. In order to achieve this, DBPA required understanding of *some* Parquet concepts. This meant one of two options for the DBPS code base: (a) take a dependency on Arrow header files, or (b) create a parallel representation for the Parquet datatypes. We selected (b).

For this reason, we created [src/common/enums.h](#) in DBPS. This header file combines these three concepts from Arrow: Type (i.e. data types, from [arrow/cpp/src/parquet/types.h](#)), CompressionCodec (from [arrow/cpp/src/arrow/util/type_fwd.h](#)) and Encoding (i.e. data/dictionary page encoding type, also from [arrow/cpp/src/parquet/types.h](#))

Per-value Encryption of Booleans in DBPA

DBPA decodes Parquet data pages to perform per-value encryption. This holds for most physical data types (INT32, INT64, FLOAT, DOUBLE, BYTE_ARRAY, etc). However, DBPA will **always perform per-block encryption for a payload** (i.e. data page) of type **BOOLEAN**.

The reasoning is twofold: First, the added net value of per-element encryption for booleans is unclear. With only two possible plaintext values (true/false) and the odds of a correct guess being 50/50, the benefit of encrypting a single boolean element is limited. Secondly, the overhead of code complexity and representation size are high. In Parquet, boolean columns are encoded as packed bitsets. The current DBPA representation of elements is aligned to bytes. Although it is possible to handle boolean values, this would result in higher payload sizes and higher code complexity.

<>